

TECHNISCHE UNIVERSITÄT CLAUSTHAL

Department of Informatics

Thesis Proposal

Adapting Small Language Models for Program Synthesis to Solve ARC-AGI

Submitted by:	Sergio Peral
Matriculation Number:	603951
Study Program:	Digital Technologies
Supervisor:	Tobias Sesterhenn
Second Examiner:	(placeholder)
Date:	January 9, 2026

Contents

1	Theme	2
2	Motivation	2
3	Relevance	2
4	Objective	2
5	Expected Results	2
6	State of Research	2
7	Research Questions	3
8	Hypothesis	3
9	Methodology	3
10	Resources	3
11	Timeline	4

1 Theme

This thesis investigates the adaptation of Small Language Models (SLMs) ($< 3B$ parameters) to solve the Abstraction and Reasoning Corpus (ARC-AGI). It explores the paradigm shift from probabilistic “next-token prediction” in large models to deterministic “program synthesis” in small, constrained neurosymbolic models.

2 Motivation

The “Scaling Hypothesis” is hitting a ceiling in abstract reasoning tasks. Frontier Large Language Models (LLMs) like GPT-4o struggle to exceed 50% accuracy on ARC-AGI without heavy scaffolding. This suggests a fundamental misalignment: ARC requires precise algorithmic deduction and generalization from few examples, while Transformers favor probabilistic memorization of surface statistics.

3 Relevance

Solving ARC is widely considered a proxy for reaching Artificial General Intelligence (AGI). This research is highly relevant as it proposes a resource-efficient path to AGI, moving away from environmentally costly massive models toward “neurosymbolic” and recursive architectures that leverage Test-Time Compute (TTC) rather than parameter scale.

4 Objective

The primary objective is to develop and explore the capabilities of program synthesis within constrained environments. The thesis aims to demonstrate that for domains governed by strict logic, constraining model capacity (using SLMs) forces the learning system to discover robust algorithms (e.g., recursive update rules) rather than relying on approximate retrieval.

5 Expected Results

The thesis seeks to determine whether an architecture combining the strengths of Library Learning (DreamCoder), Recursive State updates (TRM), and Evolutionary Search (SOAR) can theoretically surpass current LLM baselines on ARC. Expected results include a comprehensive technical analysis of these paradigms and a proposal for a novel, modular architecture that prioritizes “depth of reasoning” (recursion) over “breadth of knowledge” (parameters).

6 State of Research

The current research landscape is categorized into four converging paradigms, which form the basis of this study:

1. **Neurosymbolic Library Learning:** Pioneered by the DreamCoder framework [1], this approach uses “Wake-Sleep” Bayesian induction ($P(\rho \mid D, L)$) to grow a domain-specific “language of thought,” abstracting common logic into reusable primitives.
2. **Recursive Latent Synthesis:** The Tiny Recursive Model (TRM) [2] demonstrates that recursing a state vector in a small parameter model achieves state-of-the-art results on ARC-1. It proves that weight sharing across time steps enforces algorithmic generalization.
3. **Structured Instruction Tuning:** Research on OPT-350M [3] shows that SLMs can master agentic tool use if trained on rigorous “Thought-Action” traces, establishing an isomorphism between tool use and DSL program synthesis.
4. **Evolutionary Self-Improvement:** The SOAR framework [4] (2025 ARC Prize winner) treats SLMs as mutation operators in a genetic algorithm, using Hindsight Experience Replay to learn from failure, highlighting the necessity of dynamic Test-Time Training.

7 Research Questions

- *Primary:* How can Small Language Models (SLMs) be architecturally adapted to solve ARC-AGI via Program Synthesis?
- *Secondary:* Does constraining parameter count (to $< 3B$) effectively force the learning of generalizable algorithmic rules compared to large-scale models?

8 Hypothesis

- **The “Less is More” Hypothesis:** Constraining model capacity reduces overfitting to training examples and enforces the learning of invariant transition rules.
- **Synthesis $>$ Prediction:** Generating a program ρ that produces the output grid is more robust for ARC than predicting the output grid pixels directly.

9 Methodology

The research will employ a theoretical and comparative approach, analyzing existing paradigms and synthesizing their insights into a unified framework. Small-scale experimental validation of key components will be conducted using available computational resources (e.g., Google Colab or local GPUs).

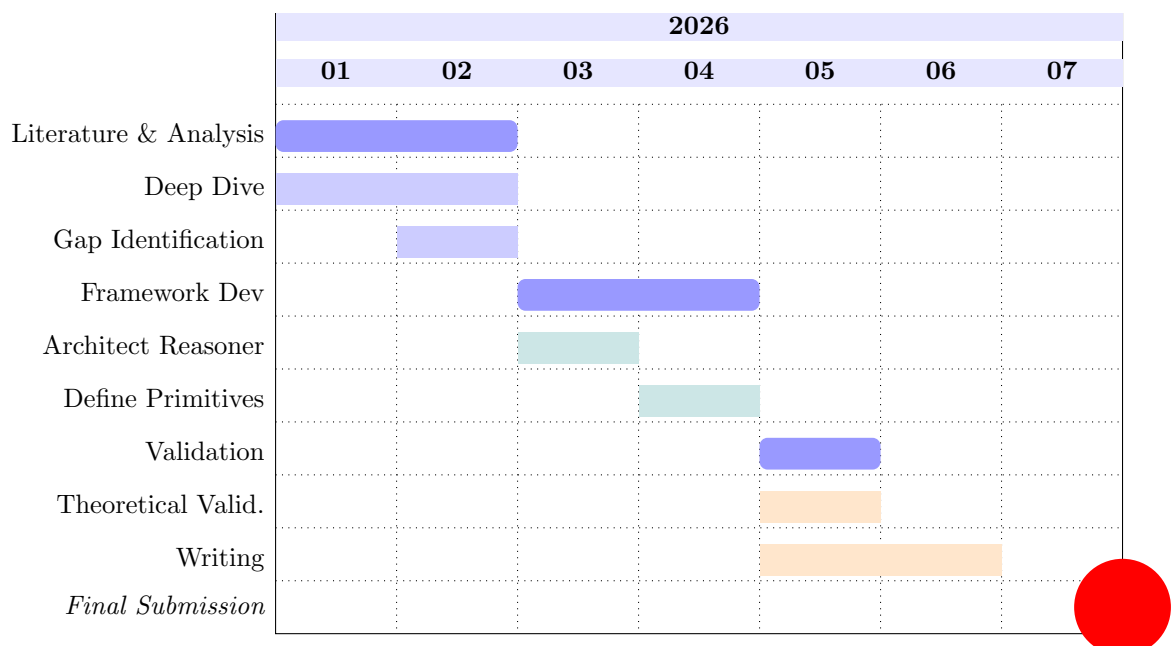
10 Resources

- Access to ARC-AGI benchmark (Evaluation Set).
- Access to technical literature and open-source repositories (GitHub) of the analyzed models.

- Computational resources (GPU) for small-scale reproduction of specific components.

11 Timeline

- **Jan. - Mar. 2026: Literature Review and Paradigm Analysis**
 - **Deep Dive into Core Papers:** Analyze the four main paradigms (Dream-Coder, TRM, OPT-350M tuning, SOAR), extracting key mathematical formulations and architectural decisions.
 - **Gap Identification:** Identify specific theoretical gaps where paradigm integration could yield improvements, documenting open questions for the framework design phase.
- **Mar. - May 2026: Framework Development**
 - **Architect the Model:** Create detailed system diagrams defining how the *Recursive Reasoner* (TRM-style) passes state vectors to the *Instruction-Tuned Decoder* (OPT-style).
 - **Define the Logical Primitives:** Theoretically map out the “Instruction Set” required for the Decoder, selecting key primitives from the **PeARL DSL** (e.g., `find_objects`, `rotate`, `filter`).
- **May - Jun. 2026: Experimental Validation and Synthesis**
 - **Benchmark:** Evaluate the DSL coverage against a subset of the ARC training tasks.
 - **Theoretical Validation:** Compare the computational complexity of the proposed framework against standard Transformer inference.
- **Jul. 2026: Final Submission**



References

- [1] K. Ellis, C. Wong, M. Nye, M. Sablé-Meyer, L. Morales, L. Hewitt, L. Cary, A. Solar-Lezama, and J. B. Tenenbaum, “Dreamcoder: Bootstrapping inductive program synthesis with wake-sleep library learning,” in *Proceedings of the 42nd acm sigplan international conference on programming language design and implementation*, 2021, pp. 835–850.
- [2] A. Jolicoeur-Martineau, “Less is more: Recursive reasoning with tiny networks,” *arXiv preprint arXiv:2510.04871*, 2025.
- [3] P. Jhandi, O. Kazi, S. Subramanian, and N. Sendas, “Small language models for efficient agentic tool calling: Outperforming large models with targeted fine-tuning,” *arXiv preprint arXiv:2512.15943*, 2025.
- [4] J. Pourcel, C. Colas, and P.-Y. Oudeyer, “Self-improving language models for evolutionary program synthesis: A case study on arc-agi,” *arXiv preprint arXiv:2507.14172*, 2025.